Guia de Especificação do Desafio Prático: Observabilidade e Pipelines Resilientes em Tempo Real

Este guia estabelece os requisitos técnicos e fundamentos teóricos para a implementação do pipeline de eventos distribuídos, focando em telemetria, resiliência e análise de performance em ambientes de alta escala.

1. Contexto e Fundamentação Teórica

1.1 Em quais sistemas reais este desafio se baseia?

Este desafio simula arquiteturas de **Event-Driven Architecture (EDA)** utilizadas por empresas de grande porte em setores como e-commerce, serviços financeiros, meios de pagamento e redes de streaming. Exemplos notáveis incluem plataformas como Uber, Netflix e Nubank.

Nessas plataformas, o fluxo principal de transações não pode depender de processamento síncrono. Em vez disso, ações — como uma compra finalizada ou uma transferência Pix — geram eventos transmitidos em tempo real para barramentos de alta vazão (como Apache Kafka) e são consumidos de forma assíncrona por múltiplos microsserviços especializados.

1.2 Por que não usar o Prometheus para armazenar eventos ou logs?

Um erro comum em arquiteturas de observabilidade é tentar enviar cada evento individual ou cada linha de log para o Prometheus. O Prometheus é um banco de dados de séries temporais (TSDB) otimizado para métricas agregadas (contadores, taxas e distribuições numéricas), utilizando um modelo de coleta por **pull**.

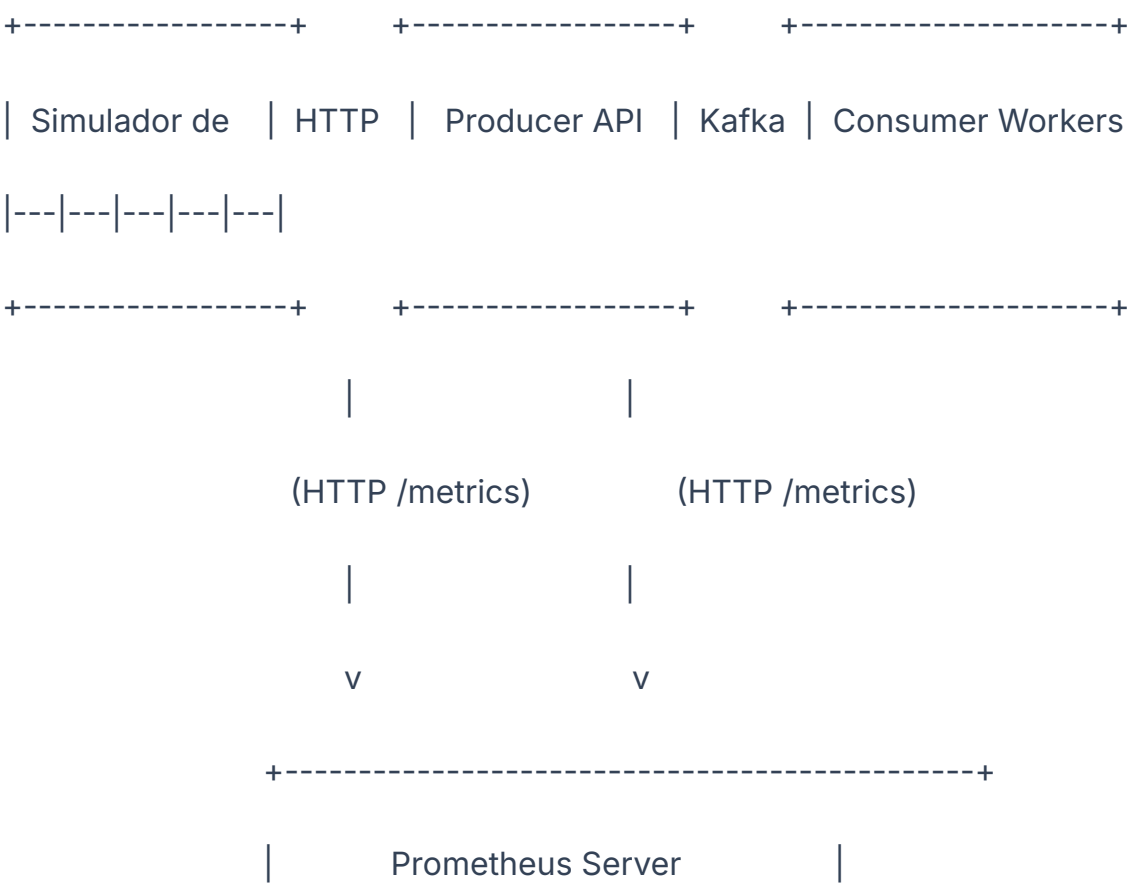
Os dados brutos dos eventos devem fluir para um armazenamento adequado (como Elasticsearch ou OpenSearch), enquanto as aplicações devem expor apenas a telemetria do seu próprio funcionamento e saúde no endpoint /metrics.

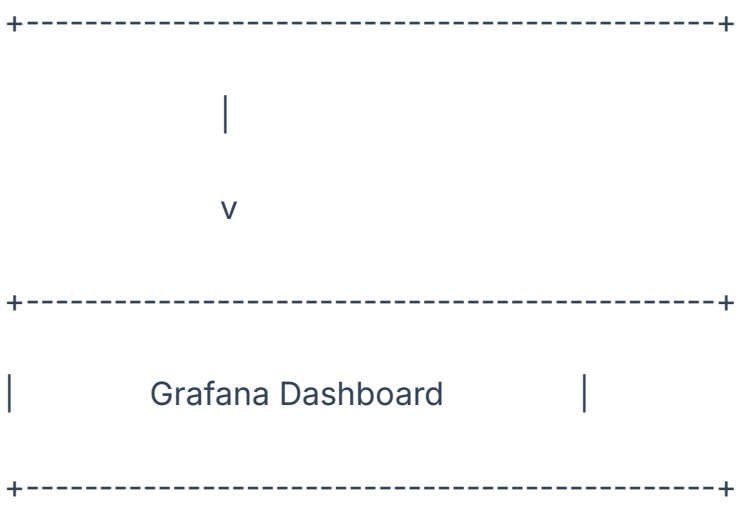
1.3 As Três Métricas RED para Microsserviços

O desafio é estruturado sobre o padrão **RED Method**, amplamente adotado pela indústria para serviços focados em requisições e eventos:

- **Rate (Taxa):** Quantidade de eventos processados por segundo (RPS/EPS).
- **Errors (Erros):** Quantidade ou percentual de falhas na ingestão, validação ou gravação.
- **Duration (Duração/Latência):** Quanto tempo cada etapa leva para ser concluída (avaliado obrigatoriamente por percentis p95 e p99).

2. Visão Geral da Arquitetura do Sistema





3. Requisitos Detalhados do Desafio

Requisito 1: Aplicação Producer (API de Ingestão)

1. Expor endpoint HTTP REST (ex.: `POST /api/v1/events`) para receber os dados de payload simulados.
2. Escrever o evento no tópico `raw-events` do Apache Kafka de forma assíncrona.
3. Expor um endpoint HTTP público `/metrics` no formato texto do Prometheus.
4. **Métricas obrigatórias:**
    - `http_requests_total` (Counter): Total de requisições recebidas (labels: `status_code`, `method`).
    - `kafka_producer_published_events_total` (Counter): Mensagens enviadas com sucesso ao Kafka.
    - `kafka_producer_errors_total` (Counter): Falhas ao tentar enviar ao Kafka.
    - `http_request_duration_seconds` (Histogram): Latência do processamento da requisição HTTP.

Requisito 2: Aplicação Consumer (Worker de Processamento)

1. Consumir mensagens do tópico `raw-events` pertencendo a um Consumer Group específico.

2. Simular um processamento com variação de tempo (ex.: simulação de gravação em banco).
3. Expor um endpoint HTTP público `/metrics` na porta configurada.
4. **Métricas obrigatórias:**
    - `kafka_consumer_processed_events_total` (Counter): Eventos processados (labels: `status` = success|error).
    - `kafka_consumer_processing_duration_seconds` (Histogram): Tempo total de processamento individual.
    - `kafka_consumer_lag_records` (Gauge): Diferença entre a última mensagem do tópico e o offset atual.

Requisito 3: Monitoramento do Cluster Kafka

1. Incluir um container auxiliar de exportação de métricas (ex.: `kafka-exporter`).
2. Coletar métricas nativas: número de partições, consumer group lag e bytes trafegados.

Requisito 4: Coleta e Configuração no Prometheus

1. O arquivo `prometheus.yml` deve conter scrape configs dinâmicos ou estáticos apontando para os serviços.
2. O intervalo de coleta (`scrape_interval`) deve ser configurado entre `2s` e `5s` para permitir reação rápida a surtos de carga.

Requisito 5: Painel Visual no Grafana (Dashboard)

O dashboard deve ser provisionado automaticamente via código e incluir:

- **Painel de Ingestão:** Gráficos de Taxa de Ingestão (req/s) e Latência em percentis p50, p95 e p99.
- **Painel de Filas:** Gráfico em tempo real de Consumer Group Lag por partição.
- **Painel de Consumo:** Gráfico da taxa de processamento vs. taxa de erros.
- **Indicador de Saúde:** Métrica de disponibilidade da aplicação.

Requisito 6: Regras de Alerta (Prometheus Alertmanager)

Criar o arquivo `alert.rules.yml` com:

1. **HighConsumerLag:** Alerta `WARNING` se o lag acumulado for > 500 mensagens por mais de 30 segundos.
2. **HighErrorRate:** Alerta `CRITICAL` se a taxa de erros do Consumer ultrapassar 5% em uma janela de 2 minutos.

4. Perguntas Frequentes (FAQ)

Q1: Por que medir latência em percentis (p95, p99) e não pela média?
A média esconde anomalias severas (tail latency). Se 99 requisições levam 10ms e 1 leva 10s, a média parece boa (~109ms), mas 1% dos clientes enfrenta uma experiência inaceitável. O p99 mostra o tempo máximo que 99% dos usuários vivenciam.

Q2: Como simular o aumento de carga (teste de estresse)?
Utilize ferramentas como K6, Locust ou Apache Bench (ab) contra o Producer para observar o comportamento dos gráficos e o crescimento da fila no Kafka.

Q3: O que acontece quando os consumidores não dão conta da carga?
O número de mensagens pendentes (*Consumer Lag*) crescerá. Se houver partições disponíveis, escalar instâncias adicionais do consumidor no mesmo grupo fará com que o Kafka rebalanceie o trabalho automaticamente.

5. Critérios de Avaliação e Entrega

Item	Descrição	Peso
Infraestrutura IAC	Subida completa do ambiente via docker-compose sem falhas.	20%

Item	Descrição	Peso
Instrumentação	Uso correto de métricas (Counters, Gauges, Histograms) e endpoint /metrics.	30%
Dashboards	Aplicação das métricas RED e clareza visual no Grafana.	20%
Alertas	Disparo correto dos alertas via Alertmanager sob estresse ou falha.	15%
Teste Prático	Diagnóstico da causa raiz de uma falha simulada durante a defesa.	15%

Aprovação técnica por: Person
Data de revisão: Date